Studio Nx

백엔드 231 — 트랙 스키마

2026-10-18

Backend(FastAPI/Python) 231 — 트랙 스키마 (만들기 · 고치기 · 응답)

항목	내용
파일	`studionx/backend/app/schemas/track.py`
작업	S1-09 기본 악기·이펙트: 트랙 요청과 응답 (PRD 02 FR-251, FR-263, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	230 디바이스 스키마 · 232 플러그인 서비스

1. 하는 일

트랙 목록·한 건·만들기·고치기의 요청과 응답 모양을 정합니다. 트랙 헤더 믹서가 다루는 칸(색, 볼륨, 팬, 뮤트, 솔로, 켜기)과 라우팅 방식, 그룹 부모, 순서가 들어 있습니다.

모델	하는 일
`TrackBase`	만들기에서 쓰는 공통 칸. 이름은 1~100자, 색은 `#RRGGBB` 형식
`TrackCreateRequest`	`TrackBase` 에 `project_id`, `order_index`, `parent_track_id` 를 더합니다
`TrackUpdateRequest`	고치기. 모든 칸이 선택이라 보낸 칸만 바뀝니다
`TrackResponse`	응답 한 건. `Track` 모델 칸을 그대로 내보냅니다
`TrackListResponse`	목록. `items` 와 `total`

2. 언제 쓰나

쓰는 곳	
`GET /projects/{id}/tracks` · `POST` (238)	목록과 만들기 요청·응답
`GET /tracks/{id}` · `PATCH` · `DELETE` (238)	한 건 조회, 고치기, 지우기
`track_service.create_track` · `update_track` (233)	요청 값을 함수 인자로 넘깁니다
화면의 트랙 헤더	색·볼륨·팬·뮤트·솔로를 이 모양으로 보냅니다

3. 코드

공통 칸과 만들기

```
TrackType = Literal["AUDIO", "MIDI", "MASTER", "GROUP", "RETURN"]
RoutingMode = Literal["CHAIN", "PATCHER"]


class TrackBase(BaseModel):
    name: str = Field(min_length=1, max_length=100)
    track_type: TrackType = "MIDI"
    color: str = Field(default="#7C5CFF", pattern=r"^#[0-9A-Fa-f]{6}$")
    volume_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
```

```
    pan: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    is_muted: bool = False
    is_solo: bool = False
    is_enabled: bool = True
    routing_mode: RoutingMode = "CHAIN"


class TrackCreateRequest(TrackBase):
    project_id: int = Field(ge=1)
    order_index: int = Field(default=0, ge=0)
    parent_track_id: int | None = None
```

고치기 (종류 `track_type` 은 바꾸지 않습니다)

```
class TrackUpdateRequest(BaseModel):
    name: str | None = Field(default=None, min_length=1, max_length=100)
    color: str | None = Field(default=None, pattern=r"^#[0-9A-Fa-f]{6}$")
    volume_db: Decimal | None = Field(default=None, ge=-60, le=24)
    pan: Decimal | None = Field(default=None, ge=-1, le=1)
    is_muted: bool | None = None
    is_solo: bool | None = None
    is_enabled: bool | None = None
    routing_mode: RoutingMode | None = None
    order_index: int | None = Field(default=None, ge=0)
    parent_track_id: int | None = None
```

응답

```
class TrackResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    track_id: int
    project_id: int
    name: str
    track_type: TrackType
    order_index: int
    color: str
    volume_db: Decimal
    pan: Decimal
    is_muted: bool
    is_solo: bool
    is_enabled: bool
    parent_track_id: int | None
    frozen_sample_id: int | None
    routing_mode: RoutingMode
    assignee_id: int | None
    locked_by: int | None
    locked_at: str | None = None
```

문법 풀이

식	뜻
`class TrackBase(BaseModel)`	공통 칸을 한 곳에 모으고 상속으로 재사용합니다
`pattern=r"^#[0-9A-Fa-f]{6}$"`	정규식 검사입니다. `#` 뒤에 16진수 6자리만 통과합니다. `red` 는 422 입니다

식	뜻
`Literal[...]` 로 종류 제한	`AUDIO`·`MIDI`·`MASTER`·`GROUP`·`RETURN` 만 허용합니다. DB ENUM 과 같습니다
`Decimal("0.00")`	소수를 정확히 다룹니다. 부동소수(0.1+0.2) 오차가 dB 계산에 끼지 않습니다
`int | None = None` (고치기)	안 보낸 칸은 None 입니다. 라우터가 `exclude_unset=True` 로 걸러 서비스에 넘기지 않습니다
`locked_at: str | None`	값이 없을 수 있는 시각 칸입니다. 지금은 화면이 쓰지 않아 문자열로 두었습니다
`ConfigDict(from_attributes=True)`	ORM 객체를 그대로 넣어도 읽습니다

4. 왜 이렇게

선택	이유
상속으로 공통 칸 재사용	만들기와 고치기의 칸 목록이 같습니다. 두 번 쓰면 한쪽만 고치는 실수가 납니다
이름을 1자 이상으로	빈 이름 트랙은 화면에서 고를 수 없습니다. 422 로 미리 막습니다
`frozen_sample_id`·`assignee_id`·`locked_by` 까지 응답에	S1-10 저장과 Phase 6 협업에서 쓸 칸입니다. 지금 넣어 두면 화면이 응답 모양을 다시 짤 필요가 없습니다
고치기에서 `track_type` 을 뺌	트랙 종류를 바꾸면 악기·이펙트 구성이 어긋납니다. 종류는 만들 때 정하고 바꾸지 않습니다
`TrackListResponse` 에 `total` 포함	화면이 "트랙 3개" 처럼 개수를 바로 보여 줍니다 (플러그인 목록과 달리 트랙은 자주 셉니다)

5. 시험

`tests/test_track_routes.py` 가 만들기 201과 기본값(`track_type` = `MIDI`, 색 `#7C5CFF`, `routing_mode` = `CHAIN`, 볼륨 `0.00`), 고치기(이름·볼륨·라우팅 방식), 형식 오류 422, 목록 순서(`order_index`, `track_id`)를 확인합니다. `tests/test_track_model.py` 는 `tracks` 칸을 실제 MySQL 과 비교합니다.